

Asignatura
Desarrollo de Software para Móviles DSM941 G01T

Experiencia de aprendizaje 3:
Foro de Discusión 1.

Nombre del Proyecto:
KAFECITO

Estudiantes:

Carlos David Martínez Ventura	**MV120141**
Carlos Alberto Alfaro Espinoza	**AE220430**
Ricardo José Aguillón González	**AG242725**
Kevin Antonio Flores Alvarado	**FA251339**
José Antonio González Oliva	**GO231492**

Fecha

18/10/2026

VIDEO DEMOSTRATIVO.

ENLACE GITHUB.

Índice.

Introducción. 3
1. Jetpack Compose: Fundamentos Técnicos. 4
1.1. Modelo Declarativo vs. Imperativo. 4
1.2. Recomposición Inteligente. 4
1.3. Gestión de Estado: remember y mutableStateOf 4
1.4. Navegación con NavHost y NavController. 5
1.5. Componentes Reutilizables y Material Design 5
2. Ventajas de Jetpack Compose Evidenciadas en el Proyecto. 6
2.1. Reducción de Código. 6
2.2. Mantenibilidad. 6
2.3. Consistencia con los Mockups. 6
2.4. Desarrollo Iterativo. 6
3. Aplicación al Proyecto Kafecito 7
Conclusiones. 9
Referencias. 10

Introducción.

Jetpack Compose representa un cambio en el desarrollo de interfaces de usuario para Android, a diferencia de otros sistemas que ha dominado la plataforma más de una década Compose adopta un enfoque declarativo donde la interfaz se describe como una función del estado de la aplicación.

Android ha establecido oficialmente que el desarrollo de nuevas aplicaciones debe ser Compose-first colocando el sistema de vistas en modo de mantenimiento y reservando las nuevas herramientas y APIs exclusivamente para compose.

Para el proyecto Kafecito esta migración no implica únicamente un cambio de herramienta sino una evolución arquitectónica que impactará directamente la forma en que el equipo realice la separación de responsabilidades coma la gestión de datos y la experiencia del usuario.

La lógica de la etapa interior debe ahora materializarse en pantallas funcionales nave gables y válidas cumpliendo con los diseños definidos en la primera etapa. En este documento buscamos justificar técnicamente cada decisión de diseño adaptada en el avance del proyecto respaldándose en características documentadas de Jetpack Compose y en su aplicación concreta al flujo de nuestra tienda.

1. Jetpack Compose: Fundamentos Técnicos.

1.1. Modelo Declarativo vs. Imperativo.

El sistema de vistas tradicional opera bajo un modelo imperativo: el desarrollador debe obtener una referencia modificar sus propiedades y gestionar manualmente su ciclo de vida. En Compose emplea un modelo declarativo donde las funciones componibles describen como debe verse la interfaz para un estado dado. Cuando el estado cambia, Compose vuelve a ejecutar las funciones afectadas un proceso denominado recomposición y actualiza únicamente los elementos que dependen de ese estado.

Esta diferencia es fundamental para nuestro proyecto, por ejemplo puede ser utilizado en el módulo de pedidos el carrito de compras ya no requiere que el desarrollador busque y modifique cada elemento de la lista manualmente en cambio el carrito se representa como una lista de datos y Compose se encarga de reflejar cualquier adición o eliminación automáticamente.

1.2. Recomposición Inteligente.

Compose Optimiza el rendimiento mediante una recomposición granular en donde sólo ejecuta nuevamente las funciones cuyos parámetros han cambiado omitiendo el resto del árbol UI. Esto es posible porque Compose rastrea qué funciones leen qué estado, estableciendo dependencias que permiten saltar componentes no afectados.

Para Kafecito esto significa cuando un empleado actualiza el estado de un pedido sólo se recomponen las secciones de la interfaz que muestran ese estado específico no toda la pantalla de gestión de pedidos.

1.3. Gestión de Estado: remember y mutableStateOf

En Compose el estado se gestiona mediante APIs observables. mutableStateOf crea un contenedor de estado que cuando es leído por una función componible establece una dependencia que dispara la recomposición cuando su valor cambia y remember preserva ese estado a través de las recomposiciones evitando que se reinicie en cada ejecución.

1.4. Navegación con NavHost y NavController.

La biblioteca navigation-compose proporciona un sistema de navegación que se integra nativamente con el modelo declarativo de compose. NavController gestiona el grafo de navegación y Navhost actúa como contenedor de los destinos. Las rutas son definidas mediante objetos serializables y los argumentos se pasan de forma tipada.

Para el proyecto la navegación entre las pantallas de autentificación coma el módulo principal y las vistas por rol se implementan definiendo rutas tales como login, register y MainMenu y pasando argumentos como el ID del usuario o el tipo de rol cuando corresponde.

1.5. Componentes Reutilizables y Material Design

Compose incluye una biblioteca completa de componentes que se pueden personalizar y cambiar. Los componentes se definen como funciones componibles pequeñas y sin estado lo que facilita su reutilización y prueba.

En Kafecito esto se traduce en la creación de componentes como ProductCard, OrderStatusBadge o InventoryItemRow, que se usa en varias pantallas como catálogo de cliente, gestión de administrador, vista de inventario de empleado.

2. Ventajas de Jetpack Compose Evidenciadas en el Proyecto.

2.1. Reducción de Código.

Estudios de casos documentados por Google indican que equipos como SoundCloud redujeron hasta un 45% las líneas de código al migrar de XML a Compose. En Kafecito, componentes como ProductCard que en XML requerirían un archivo de layout completo ahora se definen en aproximadamente 20 líneas de Kotlin.

2.2. Mantenibilidad.

Al unificar la interfaz en Kotlin, se elimina la necesidad de alternar entre archivos XML y código Kotlin. El sistema de diseño de Compose es conceptualmente más sencillo, y los componentes complejos son más fáciles de leer y modificar.

2.3. Consistencia con los Mockups.

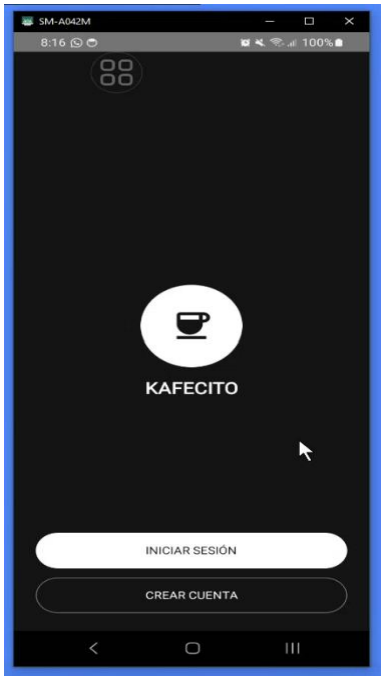
Compose facilita la implementación fiel de los mockups de Figma gracias a su sistema de temas, tipografía y componentes Material Design predefinidos. Los colores, espaciados y estilos se centralizan en un Theme de Compose, garantizando coherencia visual en toda la aplicación.

2.4. Desarrollo Iterativo.

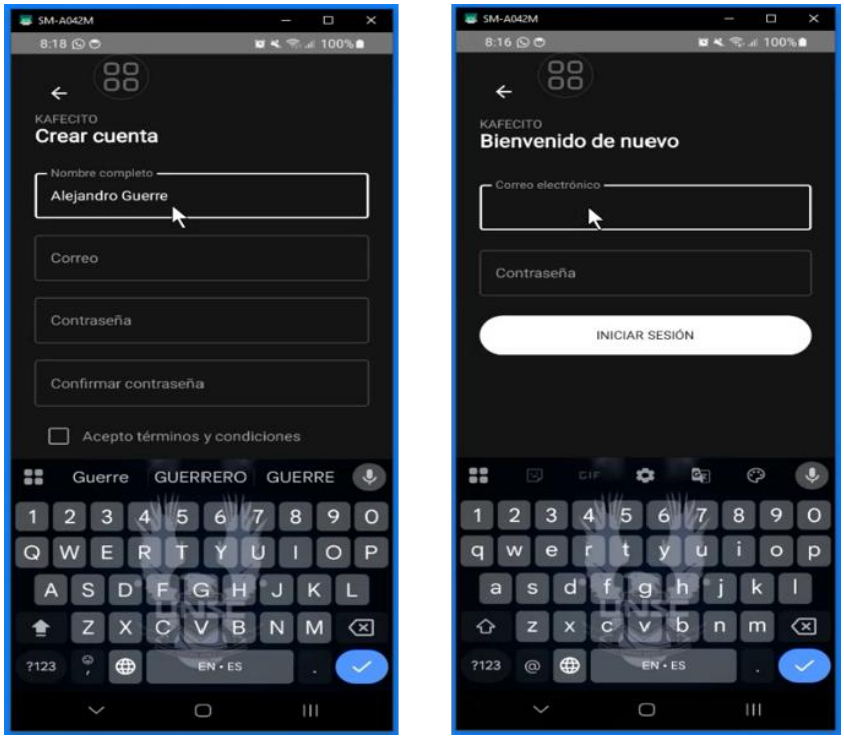
La función de vista previa en vivo de Android Studio permite visualizar los componentes sin necesidad de compilar toda la aplicación, acelerando el ciclo de diseño y validación con los mockups.

3. Aplicación al Proyecto Kafecito

Primero se muestra la pantalla de inicio que nos permite elegir entre crear una cuenta o iniciar sesión. Se implementan dos pantallas componibles, LoginScreen y RegisterScreen.

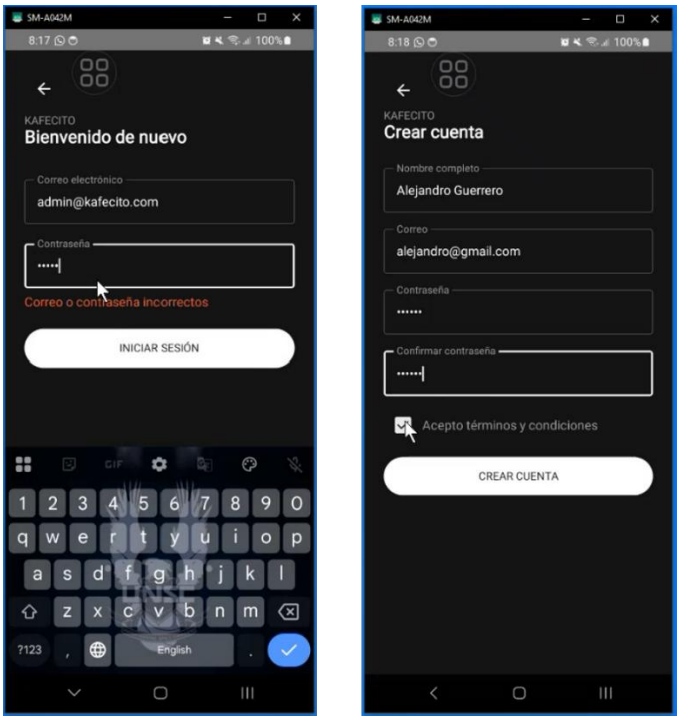


Pantalla de bienvenida.

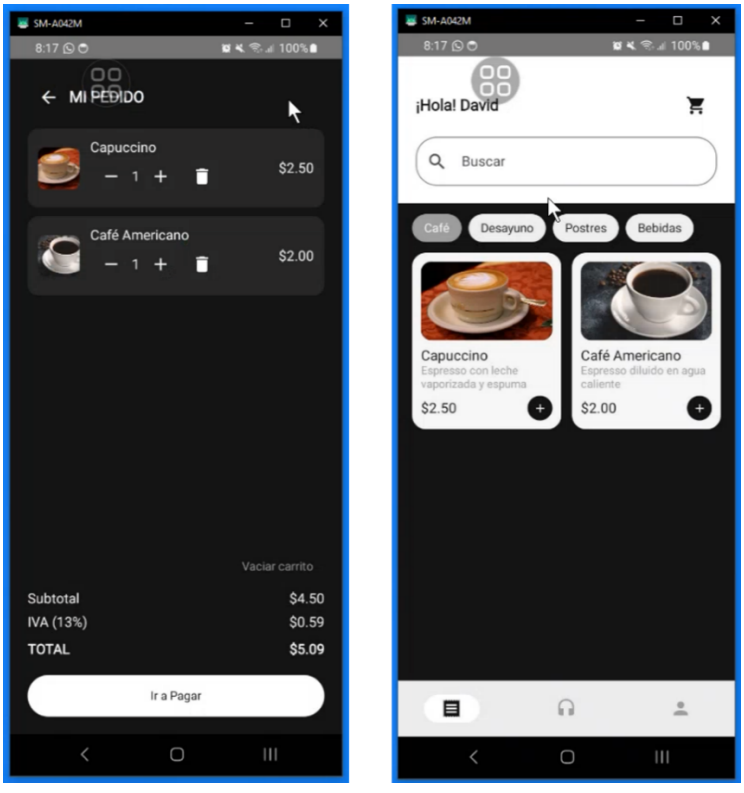


Ventana para crear cuenta y ventana para inicio de sesión.

Ventanas con las validaciones (nombre no vacío, contraseña de al menos 6 caracteres) se aplican en tiempo real y navegacion entre ventanas.



Ventana para inicio de sesión y ventana para crear cuenta con validación.



Venta de navegación entre Pantallas para carrito de compras y menú.



Ventana de perfil de usuario que permite cerrar sesión.

Conclusiones.

La migración del proyecto Kafecito hacia Jetpack Compose representa una decisión técnica fundamentada en las ventajas documentadas del framework. La construcción de las ventanas (pantallas) mediante funciones componibles ha permitido transformar la lógica de consola en una interfaz gráfica intuitiva, cumpliendo con los requerimientos de autenticación, navegación y validación.

El uso de componentes reutilizables, la gestión de estado reactiva y la navegación declarativa no solo agilizan el desarrollo, sino que garantizan que la aplicación sea escalable y fácil de mantener. Las decisiones de diseño tomadas se derivan directamente de las capacidades específicas de Compose para abordar cada necesidad funcional del proyecto, estableciendo una base sólida para la versión final del sistema.

Referencias.

- *Android Developers. (2026). Android is Compose-first. https://developer.android.com/develop/ui/compose/first*

- *Android Developers. (2026). Get started with Jetpack Compose. https://developer.android.com/develop/ui/compose/documentation*

- *Android Developers. (2026). Navegación con Compose. https://developer.android.com/develop/ui/compose/navigation*

- *Android Developers. (2026). Thinking in Compose. https://developer.android.com/develop/ui/compose/mental-model*

- *Android Developers. (2026). State in Jetpack Compose. https://developer.android.com/codelabs/jetpack-compose-state*

- *Android Developers. (2026). Por qué implementar Compose. https://developer.android.com/develop/ui/compose/why-adopt*

- *Google Developers. (2023). Kotlin para Android. https://developer.android.com/kotlin*